SERVICENOW MICRO PROJECT REPORT

Implement Client Script & UI Policy (Incident)

Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level. Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted.

Objective

The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.

Skills

Incident Management | UI Policy | UI Policy Actions | Client Scripts | Form Validation

Task 1: Create UI Policy on Incident

Activity 1: Create UI Policy

1. Log in to the ServiceNow instance with administrative or configuration access.
2. Navigate to: **System UI → UI Policies**
3. Click **New** to create a new UI Policy record.
4. Fill in the following details:
   - **Name:** High Impact Control (represents the policy purpose for high impact incidents).
   - **Table:** Incident
   - **Active:** true
5. Configure the Conditions section: Field: `Impact` | Operator: `is` | Value: `1 - High`.
6. Configure UI Policy Action for **Assignment group**: Check **Mandatory** and set **Reverse if false** to true (ensures changes revert when condition is not met).
7. Click **Submit** to save the UI Policy.

Task 2: Create UI Policy Action - Urgency

Activity 1: Configure the Urgency Field

1. Open the **High Impact Control** UI Policy and scroll down to UI Policy Actions.
2. Click **New** to create a new UI Policy Action.
3. Fill in the Action Details:
   - **Field name:** Urgency
   - **Read-only:** true (locks the field when Impact = High)
   - **Visible:** Leave alone
4. Click **Submit** to save the UI Policy Action.

Task 3: Create onChange Client Script

Activity 1: Create Client Script

1. Navigate to **System UI → Client Scripts** and click **New**.
2. Fill in details: Name: `Auto set urgency for high impact` | Table: `Incident` | Type: onChange | Field name: `Impact` | Active: `true`.
3. Paste the script below and click **Submit**:

```
function onChange(control, oldValue, newValue, isLoading) { if (isLoading || newValue == '') { return; } if (newValue == '1') { g_form.setValue('urgency', '1'); g_form.addInfoMessage('Urgency set to High for High impact incident.'); } }
```

Task 4: Create onSubmit Client Script (Save Validation)

Activity 1: Create Client Script

1. Navigate to **System UI → Client Scripts** and click **New**.
2. Fill in details: Name: `Prevent save if Assigned To missing` | Table: `Incident` | Type: onSubmit | Active: `true`.
3. Paste the script below and click **Submit**:

```
function onSubmit() { if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') { g_form.showErrorBox( 'assigned_to', 'Assigned To is mandatory for High impact incidents.' ); return false; } return true; }
```

Task 5 & 6: onCellEdit Script & Testing Activities

Task 5: Create onCellEdit Client Script

1. Create Client Script with Name: `Prevent state change via list edit` | Table: `Incident` | Type: `onCellEdit` | Field name: `State` | Active: `true`.
2. Paste script and submit:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) { alert('State cannot be updated using list editing. Please open the Incident.'); callback(false); }
```

Task 6: Testing Activities

- **Activity 1 (Mandatory Enforcement):** Set Impact to High, leave Assigned To empty, and submit. Verify that saving is prevented and an error message appears.
- **Activity 2 (Successful Save):** Select a user in Assigned To and submit. Verify successful save and auto-urgency setting.
- **Activity 3 (Reverse Condition Test):** Change Impact from High to Medium on an existing incident. Verify Assigned To is optional and Urgency becomes editable.
- **Activity 4 & 5 (List Edit & Form Update):** Verify list editing on State is blocked with an alert, while form-based State updates succeed.

Conclusion

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms, maintaining clean and consistent data.